

Zero-Downtime Modernization of an Enterprise Analytics Suite

Role: Migration & Modernization Lead · **Domain:** Legacy modernization · **Stack:** Angular 12→17, .NET Framework→.NET (modern), multi-DB

Context

An enterprise decision-support / analytics suite (multiple Angular frontends + .NET backends, several interdependent applications) was stuck on **Angular 12** and **.NET Framework**, blocking security updates, hiring, and new features. Vendor support windows were closing and the business could not tolerate a "big-bang" rewrite or visible downtime.

Problem

- Multi-hop framework jumps (**Angular 12 → 17**, and separately 8 → 14) where each major version carried breaking changes.
- Deprecated dependencies with no drop-in replacement (@angular/flex-layout, dated PrimeNG/Kendo/RxJS).
- Backend on **.NET Framework** to be moved to modern **.NET**, across **multiple database providers** (SQL Server, Oracle, PostgreSQL, MySQL).
- Toolchain drift: each Angular hop required a specific Node runtime.
- Hard constraint: **no functional regressions, no downtime, fully de-risked**.

Approach

- **Per-hop migration, not big-bang**: upgrade one major version at a time, keeping the app green at every step instead of jumping versions and debugging a wall of errors.
- **Side-by-side PRE vs POST verification**: ran the pre-migration and post-migration stacks simultaneously on isolated ports for direct visual + behavioral diffing — every screen validated against the original before sign-off.
- **Dependency remediation per hop**: migrated flex-layout to native CSS, lifted PrimeNG/Kendo/RxJS to compatible majors, pinned the correct Node runtime for each step.
- **Backend modernization with provider symmetry**: ported to modern .NET while preserving behavior across all four DB providers, with provider-specific satellite assemblies kept in lockstep.
- **Repeatable, scripted launch/stop/rebuild** of the side-by-side environment so the team could re-verify on demand rather than hand-assembling it each time (a recurring source of wasted hours).

Results

- **Angular 12 → 17** and **.NET Framework → modern .NET** completed with **no functional regressions** and **no downtime**.
- Security patch path and modern tooling **unblocked**; codebase back on supported versions.
- A **reusable migration playbook** (per-hop recipe + side-by-side harness) that turns the next app's migration from a research project into a checklist.

Why it matters to a client

Migration buyers are buying **de-risking**, not code. The differentiator here is the **side-by-side PRE/POST methodology**: the client sees that the new version behaves identically before cutover, which is what makes a board approve a modernization budget. The per-hop discipline keeps the app shippable throughout — no months-long "frozen" branch.

Edge: AI-assisted migration

I pair this with hands-on LLM/agent tooling — using AI agents to accelerate the repetitive transform work (dependency rewrites, template/API updates) while the side-by-side harness keeps verification objective. This is the fast-growing, premium end of modernization work.

Tech

Angular 8/12 → 14/17 · .NET Framework → .NET 8/10 · C# · TypeScript · RxJS · PrimeNG · Kendo · SQL Server / Oracle / PostgreSQL / MySQL · Node (per-hop) · PowerShell automation

Client/product name withheld for confidentiality. References and detailed before/after available under NDA.

Lorenzo Castrico · castricolorenzo@gmail.com · Senior Full-Stack & Modernization Engineer